



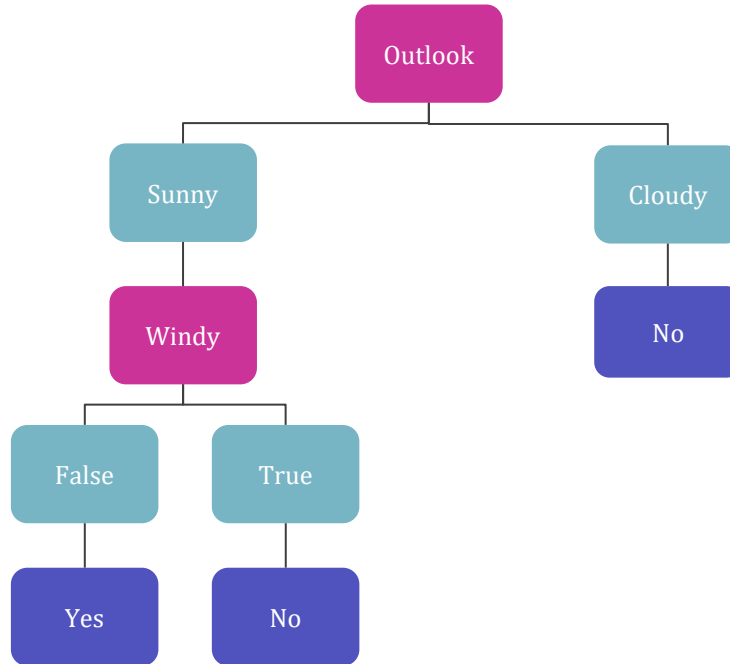
QpiAI™ - Joint AI Training and Certification Program

Decision Trees, Random Forest and Ensemble
Techniques

Sachin Kumar

Motivation for Decision Tree

Whether to Play Tennis on a certain day?



Play Tennis =

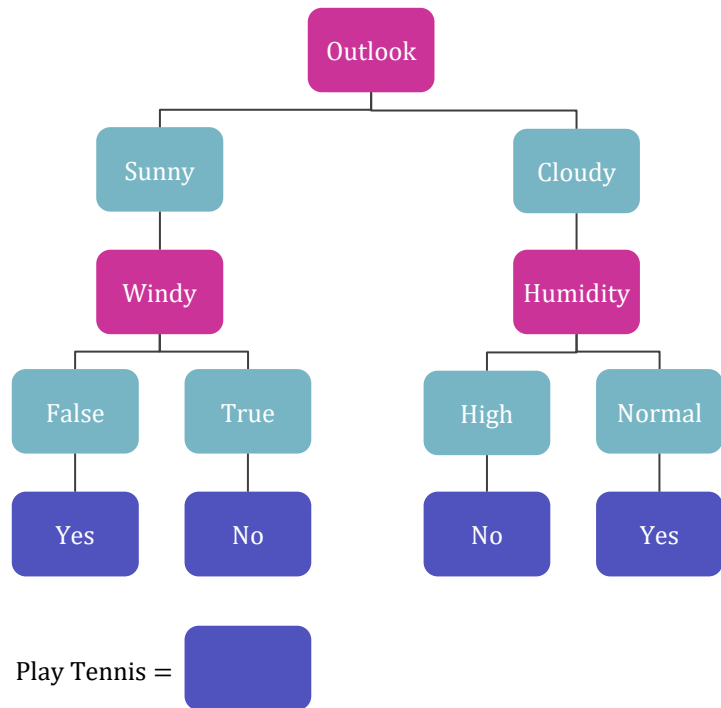


Decision Tree Example

Predictors				Target
Outlook	Temp	Humidity	Windy	Play Tennis
Cloudy	Cool	High	False	No
Cloudy	Hot	High	False	No
Sunny	Hot	Normal	False	Yes
Sunny	Cool	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Sunny	Cool	High	False	Yes
Cloudy	Mild	High	False	No
Cloudy	Cool	Normal	False	Yes
Sunny	Mild	Normal	True	No
Cloudy	Mild	Normal	True	Yes
Sunny	Hot	High	False	Yes
Sunny	Mild	Normal	False	Yes
Sunny	Mild	High	False	Yes



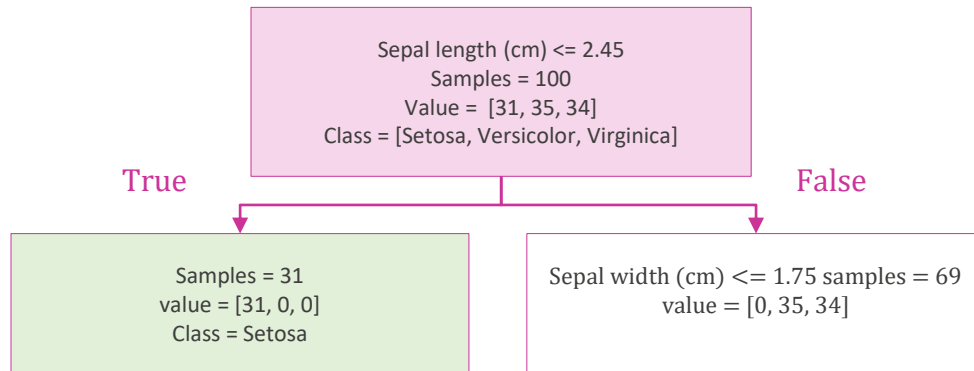
Decision Tree

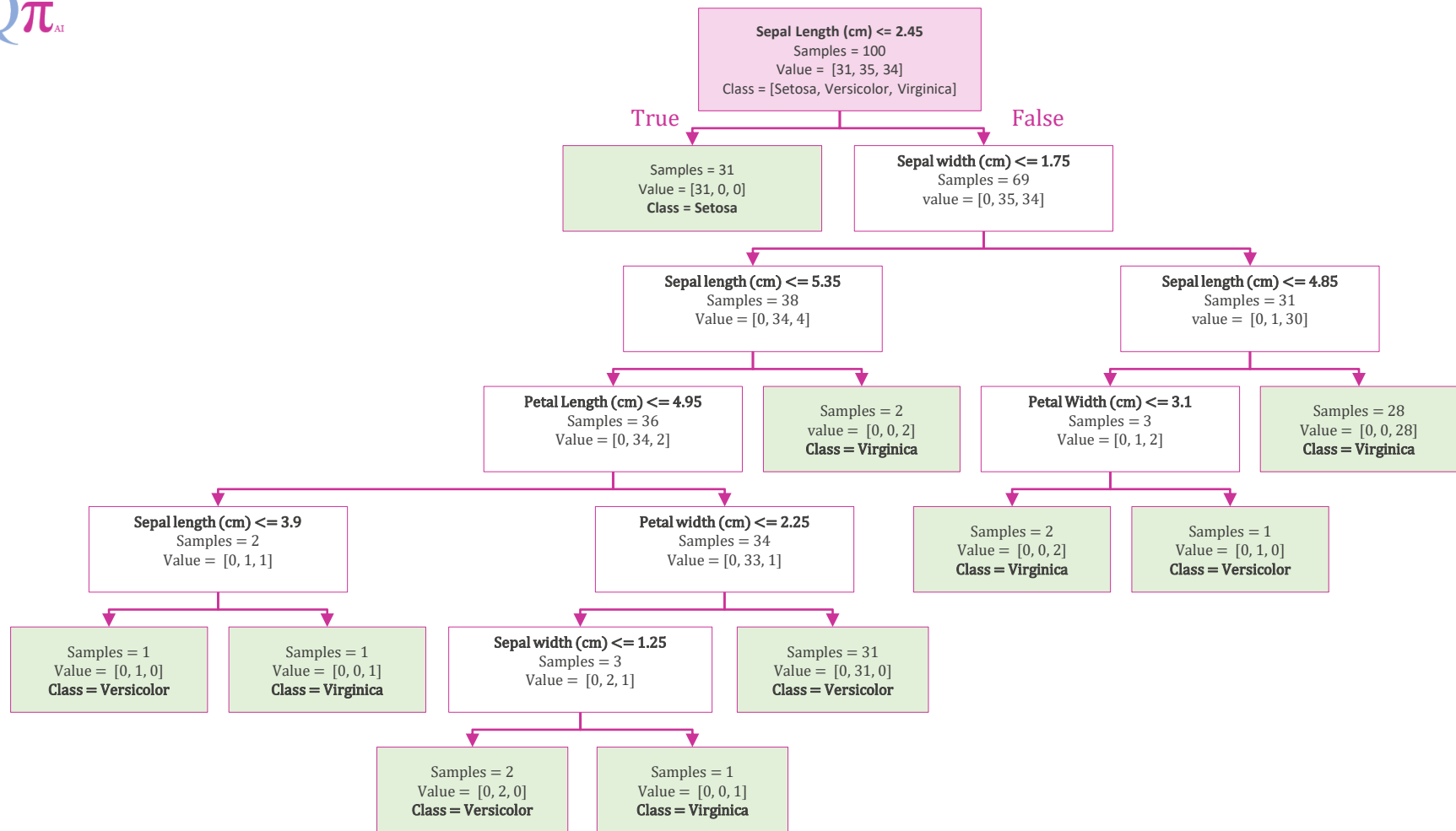


Decision Tree

What are Decision Trees?

- A tree based supervised learning algorithm.
- Each parent node here represents a choice to split data based on a condition.
- Each leaf node represents a decision in case of a stump OR a series of decisions in case of a tree.
- Solves both classification and regression problems.



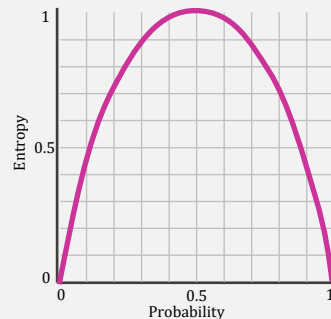


Entropy is defined as the degree of randomness in a system.

Given a categorical feature S , containing ' c ' different classes, the entropy of S relative to this classification:

$$\text{Entropy} = \sum_{i=1}^c -p_i * \log_2(p_i)$$

Where p_i is the probability of a random sample belonging to class i



It takes up a value close to 0 if all p_i 's are close to 0 or 1, that is the categorical feature S being dominated by a single class.

Information Gain

- Decides which feature goes into a decision node.
- To minimize the decision tree depth, the feature with the most entropy reduction is the best choice.

The information gain, $\text{Gain}(S,A)$ of a feature A ,

$$\text{Gain}(S,A) = \underbrace{\text{Entropy}(S)}_{\text{Original entropy of } S} - \underbrace{\sum_{v \in \text{values}(A)} \frac{|S_v|}{|S|} \cdot \text{Entropy}(S_v)}_{\text{Relative entropy of } S}$$

Where:

S is value v of all possible values of feature A

S_v = subset of S for which feature A has value v

$|S_v|$ = number of elements in S_v

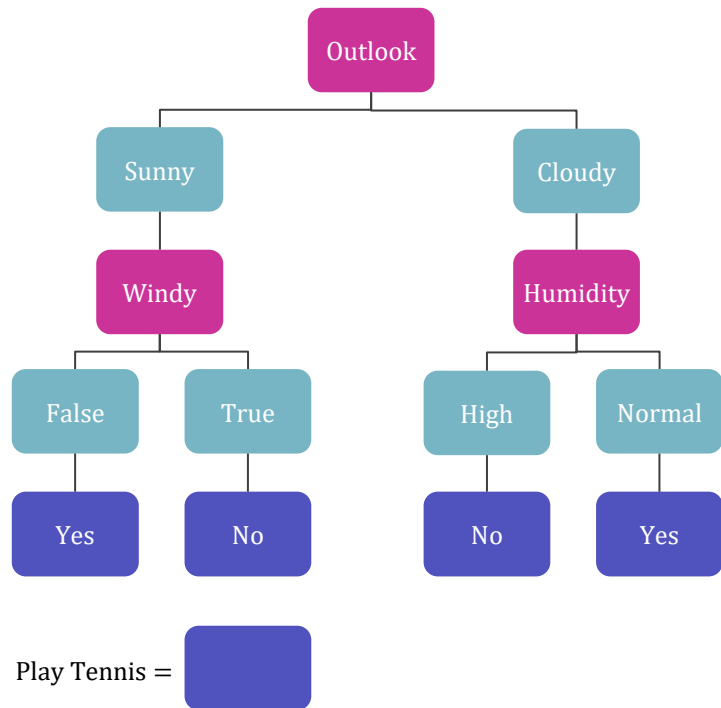
$|S|$ = number of elements in S

Decision Tree Example

Predictors				Target
Outlook	Temp	Humidity	Windy	Play Tennis
Cloudy	Cool	High	False	No
Cloudy	Hot	High	False	No
Sunny	Hot	Normal	False	Yes
Sunny	Cool	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Sunny	Cool	High	False	Yes
Cloudy	Mild	High	False	No
Cloudy	Cool	Normal	False	Yes
Sunny	Mild	Normal	True	No
Cloudy	Mild	Normal	True	Yes
Sunny	Hot	High	False	Yes
Sunny	Mild	Normal	False	Yes
Sunny	Mild	High	False	Yes



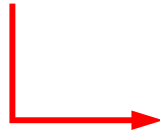
Decision Tree



Entropy Calculation

$$E(S) = \sum_{i=1}^c -p_i \log_2 p_i$$

Play Tennis	
Yes	No
9	5



Entropy (Play Tennis) = Entropy (9,5)
= Entropy (9/(5 + 9), 5/(5 + 9))
= Entropy (9/14, 5/14)
= - ((9/14) log₂ (9/14)) - ((5/14) log₂ (5/14))
= 0.94

Information Gain Calculation for a Feature

$$E(T, X) = \sum_{c \in X} P(c) E(c)$$

		Play Tennis		
		Yes	No	
Outlook	Sunny	7	2	9
	Cloudy	2	3	5
		9	5	14

$$\begin{aligned}
 E(\text{Play Tennis, Outlook}) &= P(\text{Sunny}) * E(7,2) + P(\text{Cloudy}) * E(2,3) \\
 &= (9/14) * 0.76 + (5/14) * 0.97 \\
 &= 0.838
 \end{aligned}$$

$$\text{Gain}(T, X) = \text{Entropy}(T) - \text{Entropy}(T, X)$$

$$\begin{aligned}
 G(\text{Play Tennis, Outlook}) &= E(\text{Play Tennis}) - E(\text{Play Tennis, Outlook}) \\
 &= 0.940 - 0.838 = 0.102
 \end{aligned}$$

Information Gain Calculation

		Play Tennis	
		Yes	No
Outlook	Sunny	7	2
	Cloudy	2	3
Gain = 0.102			

		Play Tennis	
		Yes	No
Temp.	Hot	2	1
	Mild	3	2
	Cool	4	2
Gain = 0.003			

		Play Tennis	
		Yes	No
Humidity	High	4	3
	Normal	5	2
Gain = 0.016			

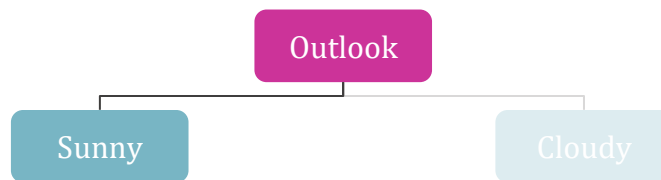
		Play Tennis	
		Yes	No
Windy	False	7	3
	True	2	2
Gain = 0.079			

Node is made for Feature with highest Information Gain

Outlook		Temp	Humidity	Windy	Play Tennis
Sunny	Sunny	Hot	Normal	False	Yes
	Sunny	Cool	High	False	Yes
	Sunny	Cool	Normal	False	Yes
	Sunny	Cool	Normal	True	No
	Sunny	Cool	High	False	Yes
	Sunny	Mild	Normal	True	No
	Sunny	Hot	High	False	Yes
	Sunny	Mild	Normal	False	Yes
	Sunny	Mild	High	False	Yes
	Cloudy	Cool	High	False	No
Cloudy	Cloudy	Hot	High	False	No
	Cloudy	Mild	High	False	No
	Cloudy	Cool	Normal	False	Yes
	Cloudy	Mild	Normal	True	Yes

Further splits and Leaf Node Condition

Temp	Humidity	Windy	Play Tennis
Hot	Normal	False	Yes
Cool	High	False	Yes
Cool	Normal	False	Yes
Cool	Normal	True	No
Cool	High	False	Yes
Mild	Normal	True	No
Hot	High	False	Yes
Mild	Normal	False	Yes
Mild	High	False	Yes



$$\begin{aligned}\text{Entropy (Play Tennis)} &= \text{Entropy (7,2)} \\ &= 0.764\end{aligned}$$

$$\begin{aligned}G(\text{Play Tennis, Temp}) &= E(\text{Play Tennis}) - E(\text{Play Tennis, Temp}) \\ &= 0.764 - 0.666 \\ &= 0.098\end{aligned}$$

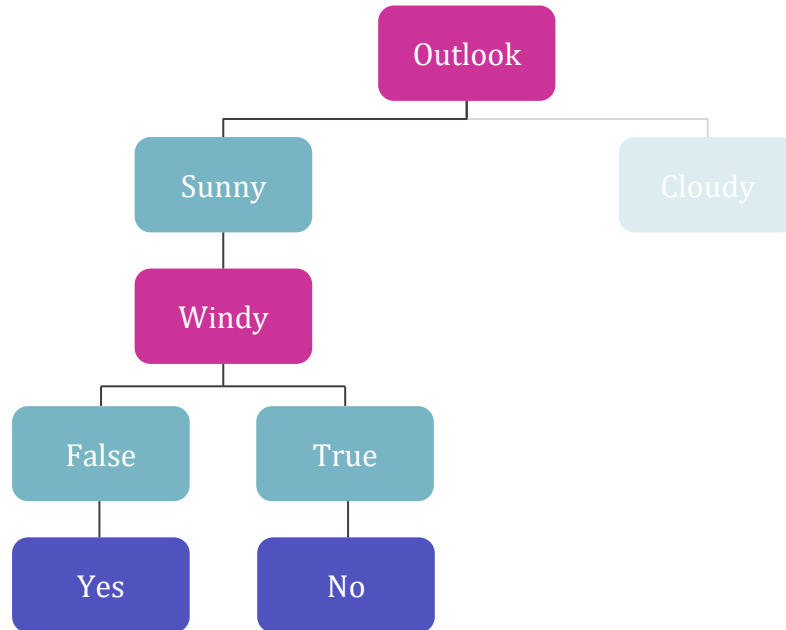
$$\begin{aligned}G(\text{Play Tennis, Humidity}) &= E(\text{Play Tennis}) - E(\text{Play Tennis, Humidity}) \\ &= 0.764 - 0.539 \\ &= 0.225\end{aligned}$$

$$\begin{aligned}G(\text{Play Tennis, Windy}) &= E(\text{Play Tennis}) - E(\text{Play Tennis, Windy}) \\ &= 0.764 - 0 \\ &= \mathbf{0.764}\end{aligned}$$

Further splits and Leaf Node Condition

Temp	Humidity	Windy	Play Tennis
Hot	Normal	False	Yes
Cool	High	False	Yes
Cool	Normal	False	Yes
Cool	High	False	Yes
Hot	High	False	Yes
Mild	Normal	False	Yes
Mild	High	False	Yes

Temp	Humidity	Windy	Play Tennis
Cool	Normal	True	No
Mild	Normal	True	No



Play Tennis =

Further splits and Leaf Node Condition

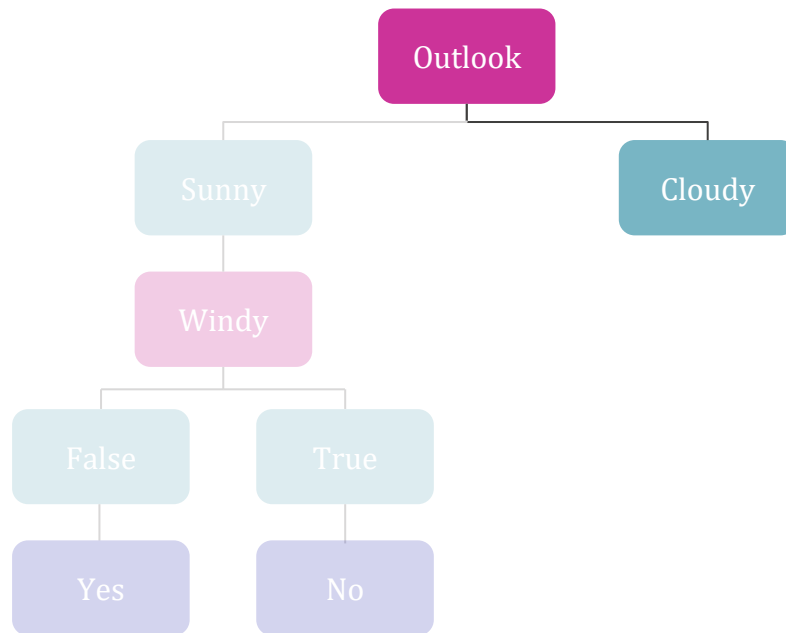
Temp	Humidity	Windy	Play Tennis
Cool	High	False	No
Hot	High	False	No
Mild	High	False	No
Cool	Normal	False	Yes
Mild	Normal	True	Yes

$$\begin{aligned}\text{Entropy (Play Tennis)} &= \text{Entropy (2,3)} \\ &= 0.971\end{aligned}$$

$$\begin{aligned}G(\text{Play Tennis, Temp}) &= E(\text{Play Tennis}) - E(\text{Play Tennis, Temp}) \\ &= 0.971 - 0.8 \\ &= 0.171\end{aligned}$$

$$\begin{aligned}G(\text{Play Tennis, Humidity}) &= E(\text{Play Tennis}) - E(\text{Play Tennis, Humidity}) \\ &= 0.971 - 0 \\ &= \mathbf{0.971}\end{aligned}$$

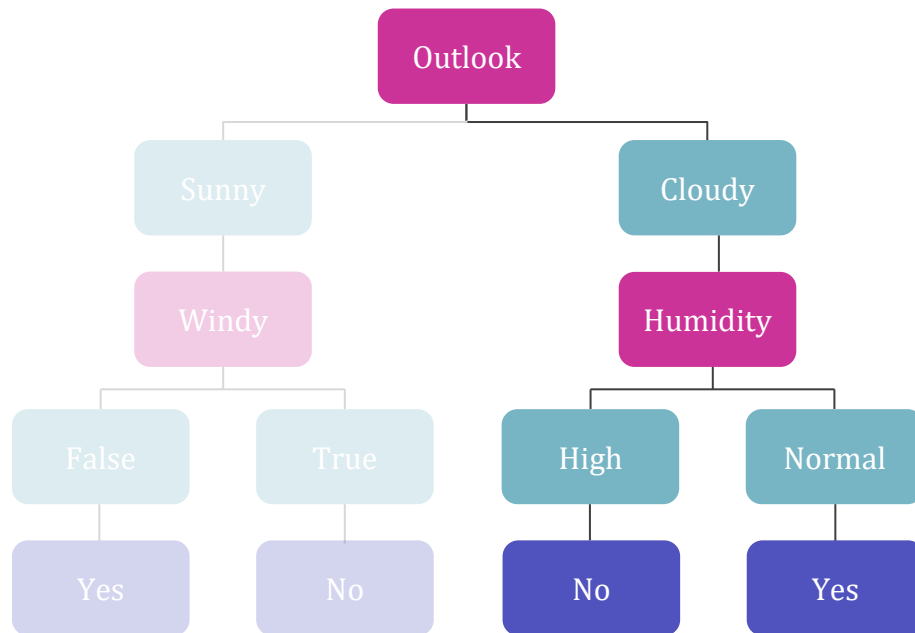
$$\begin{aligned}G(\text{Play Tennis, Windy}) &= E(\text{Play Tennis}) - E(\text{Play Tennis, Windy}) \\ &= 0.971 - 0.649 \\ &= 0.322\end{aligned}$$



Further splits and Leaf Node Condition

Temp	Humidity	Windy	Play Tennis
Cool	High	False	No
Hot	High	False	No
Mild	High	False	No

Temp	Humidity	Windy	Play Tennis
Cool	Normal	False	Yes
Mild	Normal	True	Yes

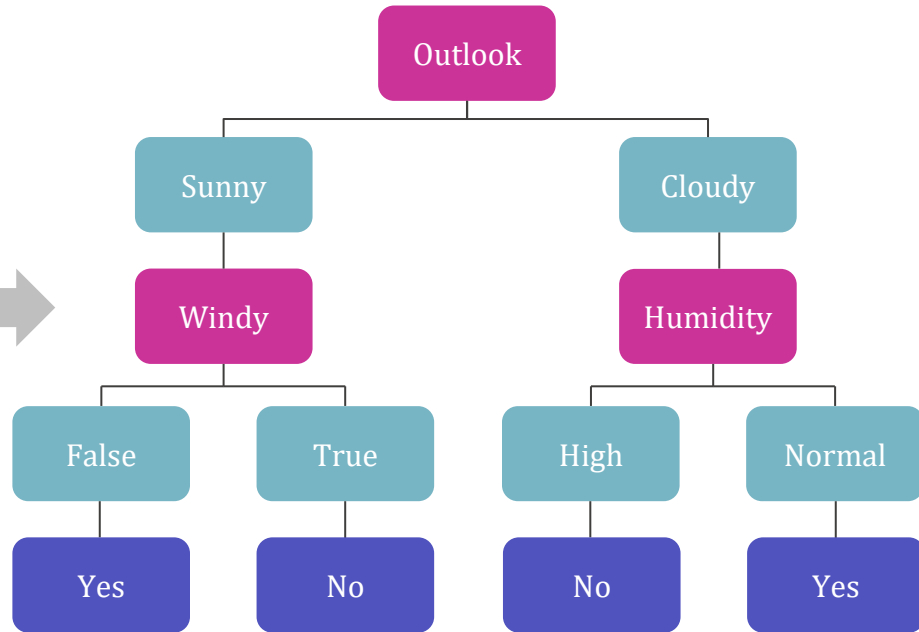


Play Tennis =



Final Decision Tree

Outlook	Temp	Humidity	Windy	Play Tennis
Cloudy	Cool	High	False	No
Cloudy	Hot	High	False	No
Sunny	Hot	Normal	False	Yes
Sunny	Cool	High	False	Yes
Sunny	Cool	Normal	False	Yes
Sunny	Cool	Normal	True	No
Sunny	Cool	High	False	Yes
Cloudy	Mild	High	False	No
Cloudy	Cool	Normal	False	Yes
Sunny	Mild	Normal	True	No
Cloudy	Mild	Normal	True	Yes
Sunny	Hot	High	False	Yes
Sunny	Mild	Normal	False	Yes
Sunny	Mild	High	False	Yes



Play Tennis =



Gini Index

- It is the measure of total variance across the all the 'c' classes.
- Like Entropy, it also takes up a value close to 0 if all p_i 's are close to 0 or 1, that is the collection being dominated by a single class.
- Gini Index is referred to as a measure for Node Purity.

$$G = \sum_{k=1}^c p_k(1 - p_k)$$

- It is computationally less expensive and quicker to calculate as compared to Entropy because it is simple algebraic equations without any logarithmic functions.
- It is generally used when handling a very large feature space with limited computation power and time-constraints.

Pros and Cons of Decision Trees



- Can handle huge datasets
- Can handle mixed predictors—categorical and continuous
- Easily ignore redundant variables
- Small trees are easy to interpret



- Very large trees are hard to interpret
- Prediction performance is often poor as it is prone to overfitting

Pruning - Hyperparameters

Maximum Depth

- The deeper the tree, the more splits it has & it captures more information about the data.
- However, in general a decision tree overfits for large depth values. The tree perfectly predicts all the train data; however, it fails to generalize the findings for new data.

Minimum Sample Split

- Minimum Sample Split specifies the minimum number of samples required to split an internal node.
- We can either specify a number to denote the minimum number or a fraction to denote the percentage of samples in an internal node.

Minimum Sample Leaf

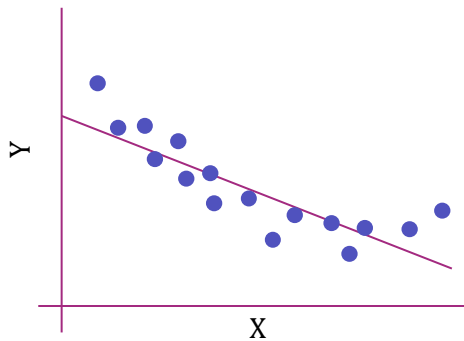
- Minimum Sample Leaf is the minimum number of samples required to be at a leaf node.
- This parameter is like Minimum Sample Split; however, this describes the minimum number of samples at the leaves, i.e. the base of the tree.

A comparison between Minimum Sample Split and Minimum Sample Leaf

- The main difference between the two is that Minimum Sample Leaf guarantees a minimum number of samples in a leaf, while Minimum Sample Split can create arbitrary small leaves.
- For instance, if Minimum Sample Split = 5, & there are 7 samples at an internal node then the split is allowed. But let's say the split results in two leaves, one with 1 sample, and another with 6 samples.
If Minimum Sample Leaf = 2, then the split won't be allowed (even if the internal node has 7 samples) because one of the leaves resulted will have less than the minimum number of samples required to be at a leaf node.

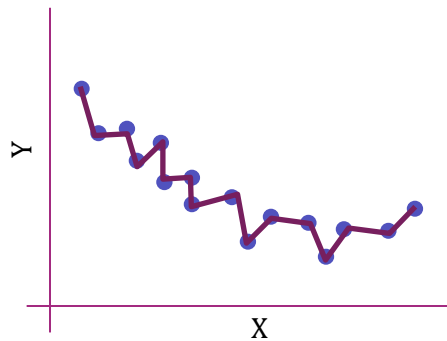
Overfit and Underfit (Revisited)

Underfit (High Bias)



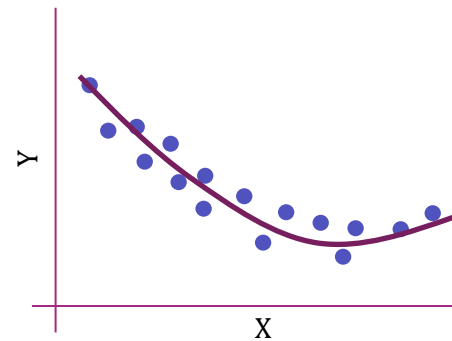
$$Y = \theta_0 + \theta_1 * X$$

Overfit (High Variance)

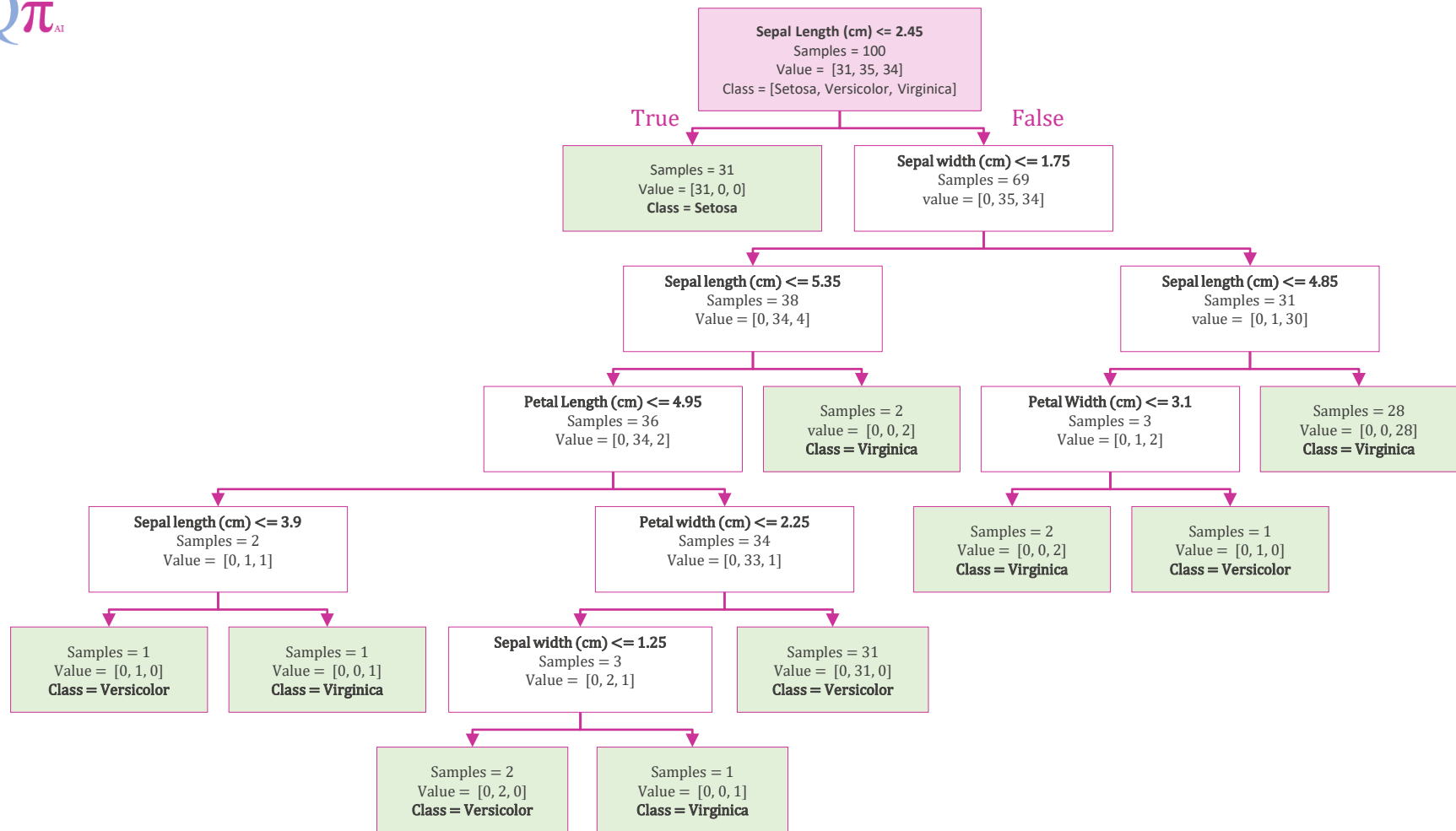


$$Y = \theta_0 + \theta_1 * X + \theta_2 * X^2 + \theta_3 * X^3 + \theta_4 * X^4$$

Balanced fit

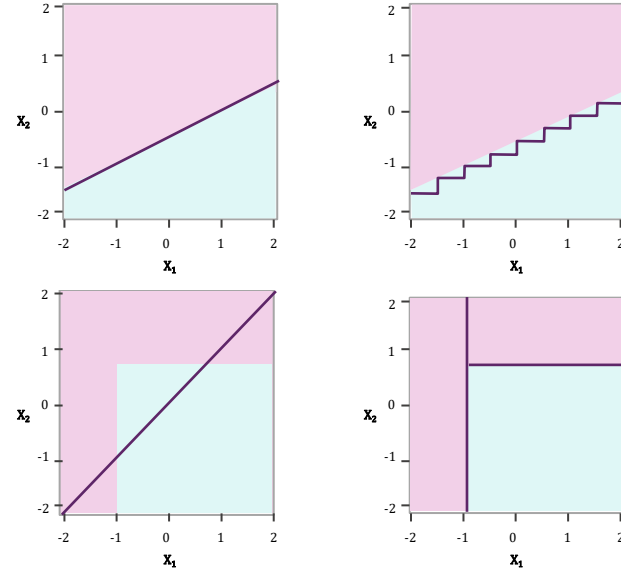
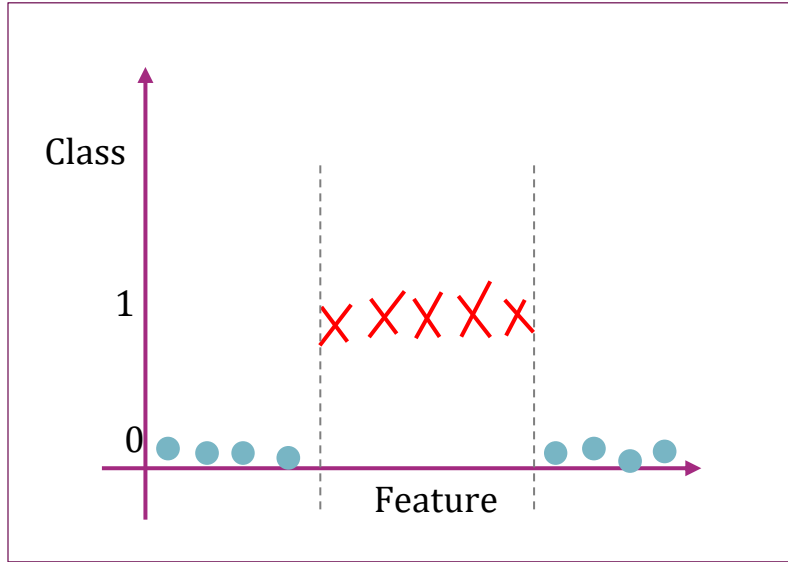


$$Y = \theta_0 + \theta_1 * X + \theta_2 * X^2$$



Comparison of Decision Trees with SVM

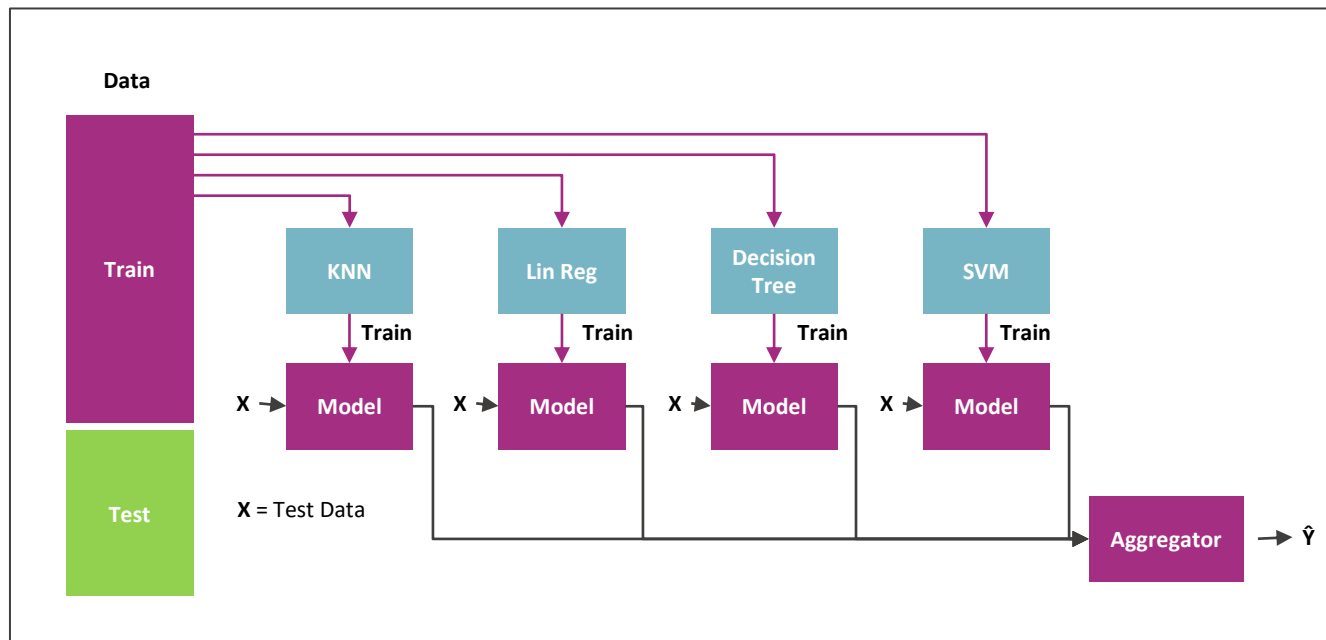
Linear models can't handle the cases where there are multiple ranges for classes or the decision boundary is non-linear. Decision tree can successfully handle those scenarios.



A two dimensional classification example in which the true decision boundary is linear, and is indicated by the shaded regions. A classical approach that assume a linear boundary (left) will outperform a decision tree that performs splits parallel to the axes (right). Bottom Row: Here the true decision boundary is non-linear. Here a linear model is unable to capture the true decision boundary (left), whereas a decision tree is successful (right)

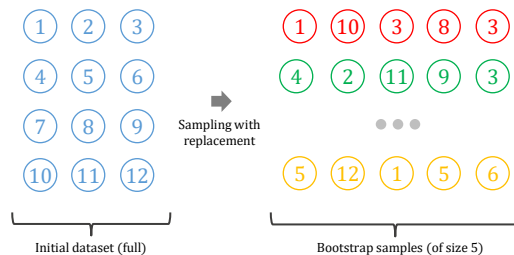
Ensemble

- Single “weak learners” have the problem of their individual bias or variance.
- So multiple “weak learners” are combined to form one strong model which has a lower bias and variance than all its individual building blocks.
- If all the “weak learners” are same type of model, then it’s called Homogeneous Ensemble(eg: Random Forest) and if the “weak learners” are different then it’s called Heterogeneous Ensemble.



Bagging – Bootstrap Aggregating

Bootstrapping is a statistical technique consists in generating samples of size B (called bootstrap samples) from an initial dataset of size N by randomly drawing with replacement B observations.

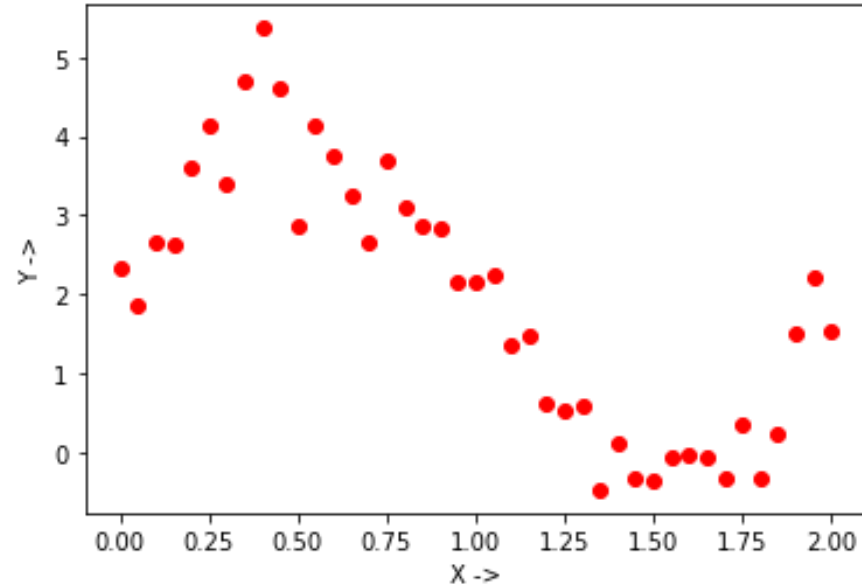


We train a model for each of the different bootstrap samples i.e. $f_1(x)$, $f_2(x)$, ..., $f_K(x)$ and take the average of them

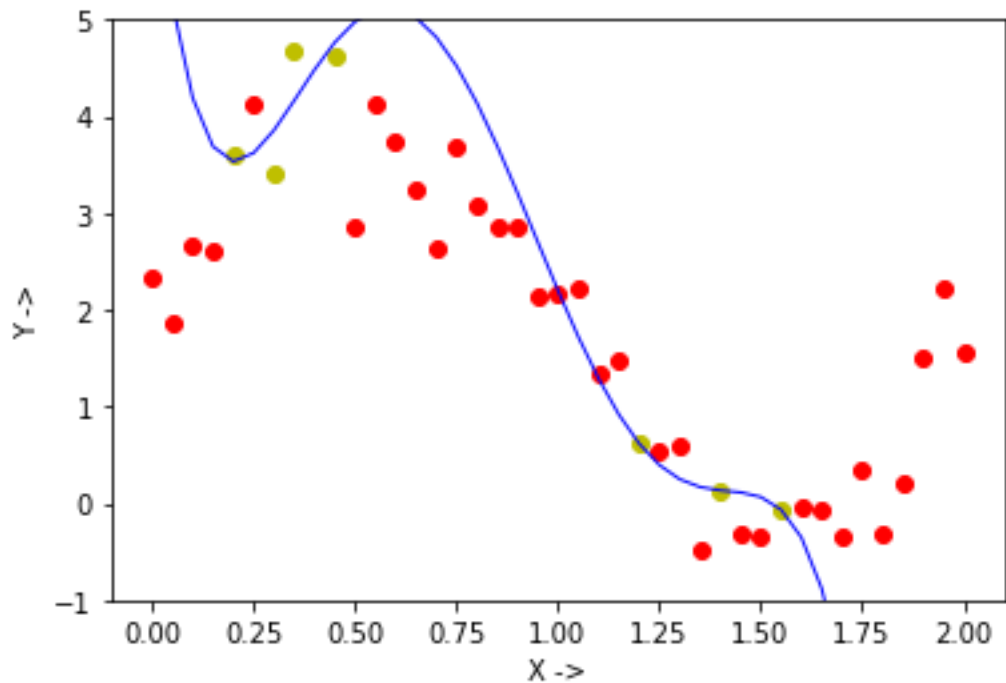
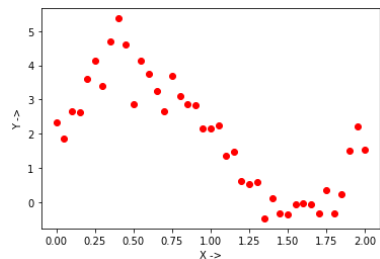
$$\hat{f}_{\text{avg}}(x) = \frac{1}{K} \sum_{b=1}^K f_b(x)$$

Bagging can be parallelised by parallelly selecting the bootstrap sample and training the “weak learners” parallelly.

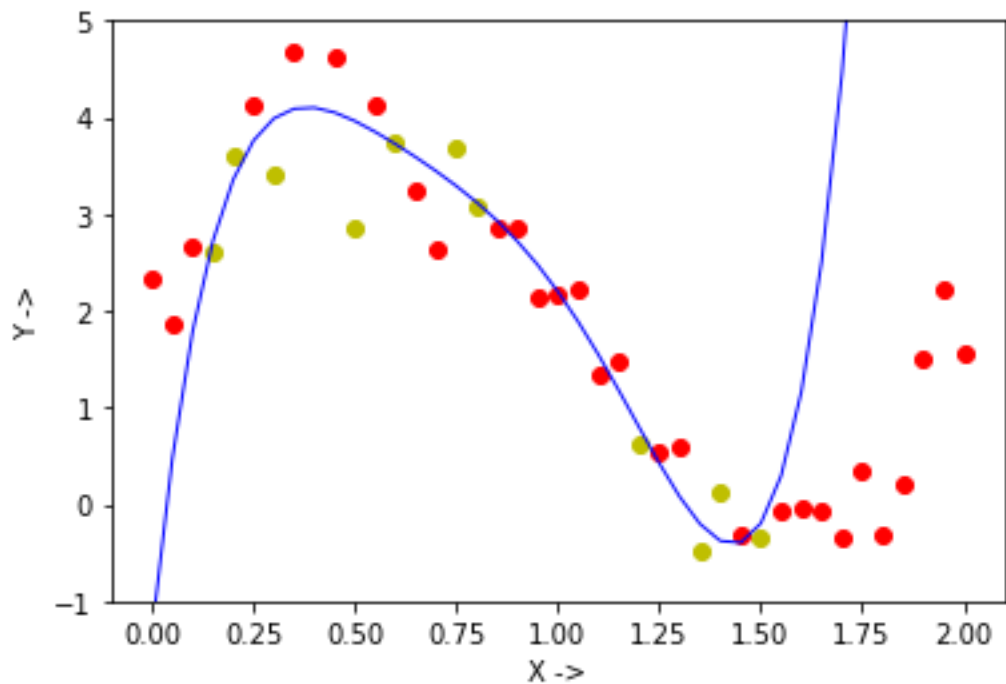
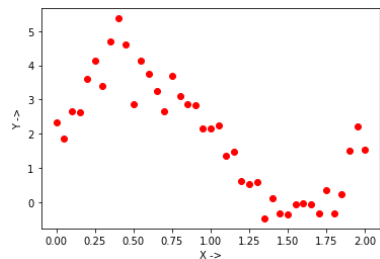
Bagging Example



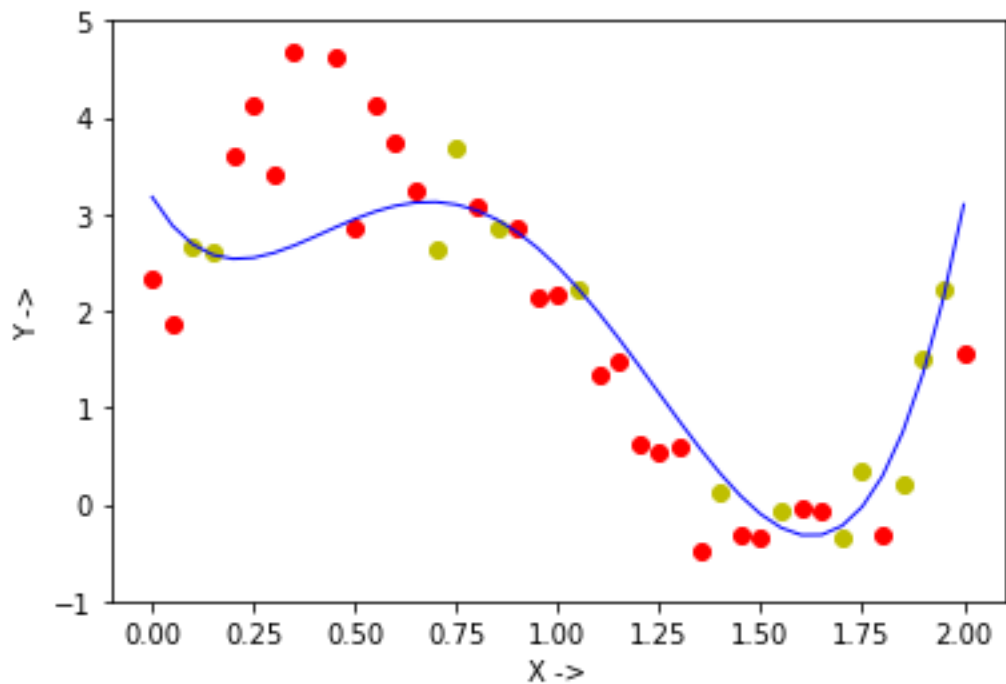
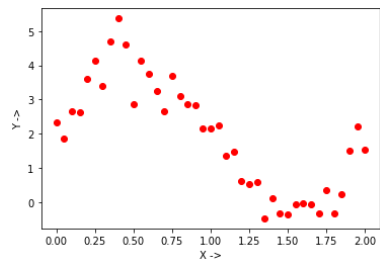
Bagging Example



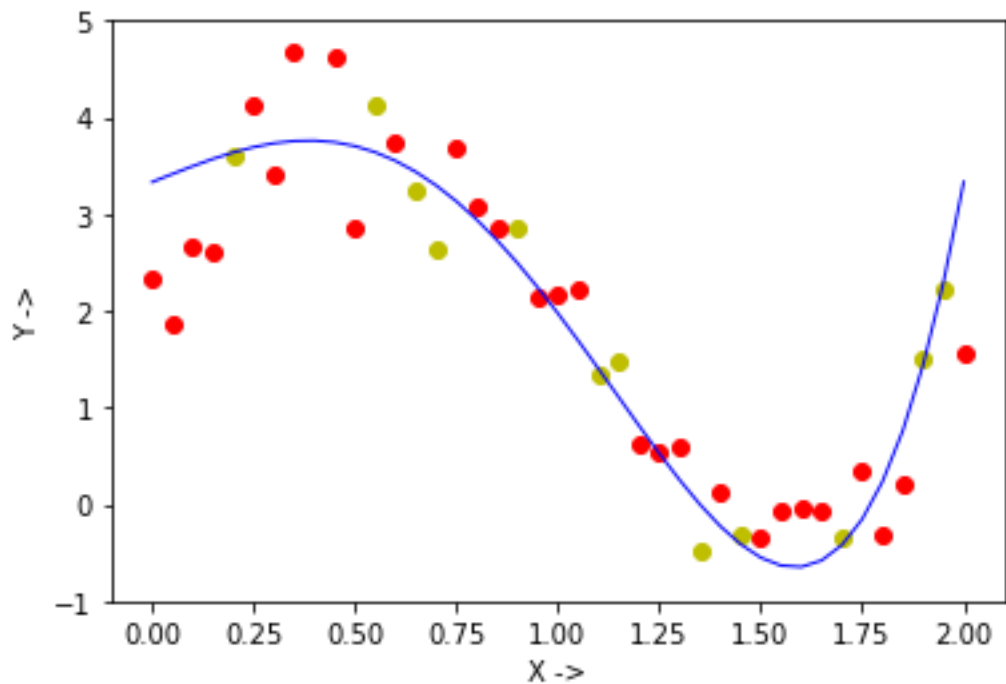
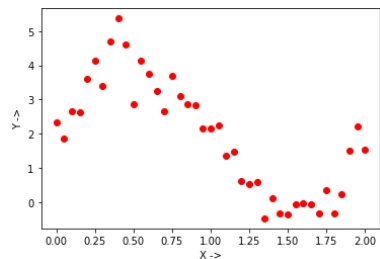
Bagging Example



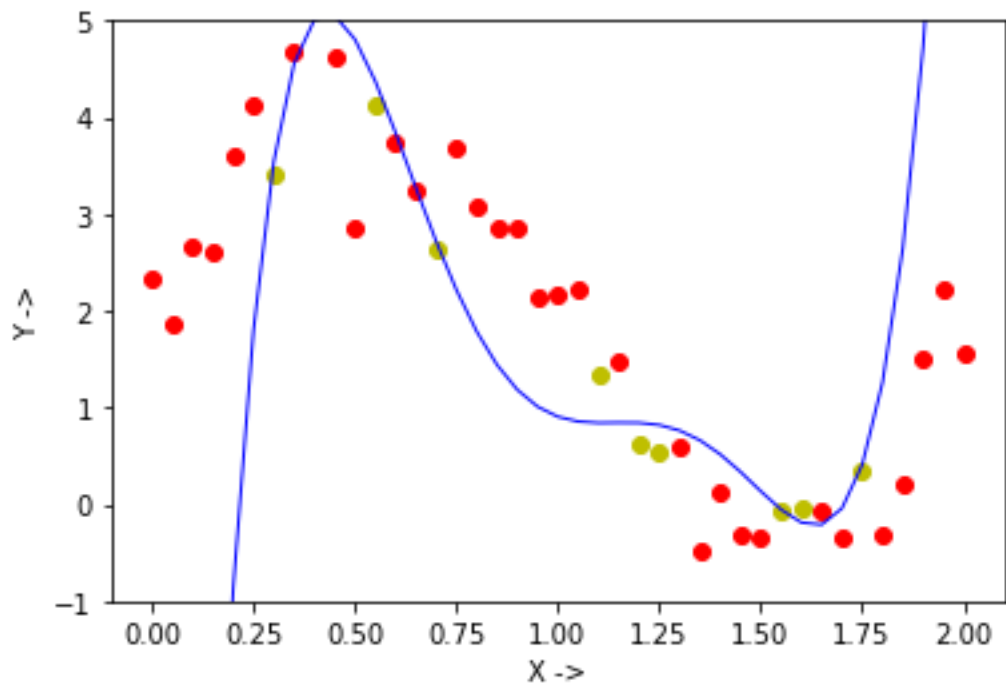
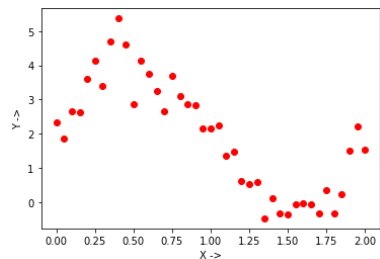
Bagging Example



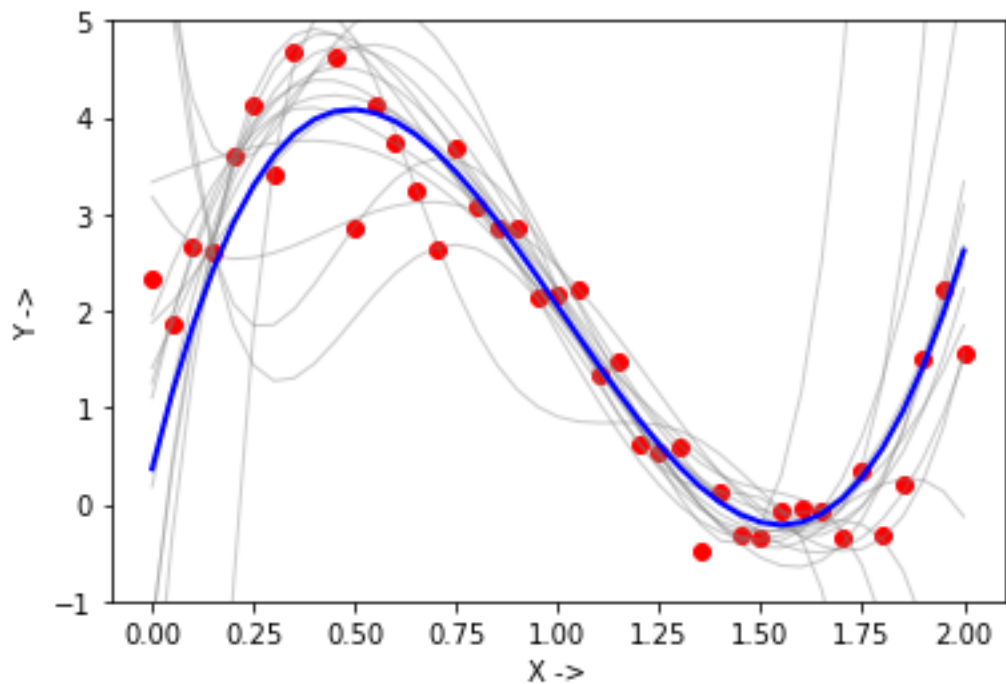
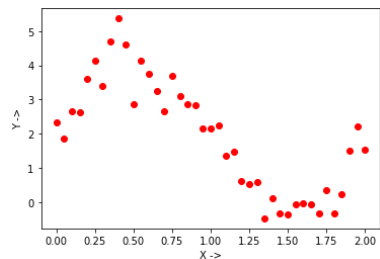
Bagging Example



Bagging Example

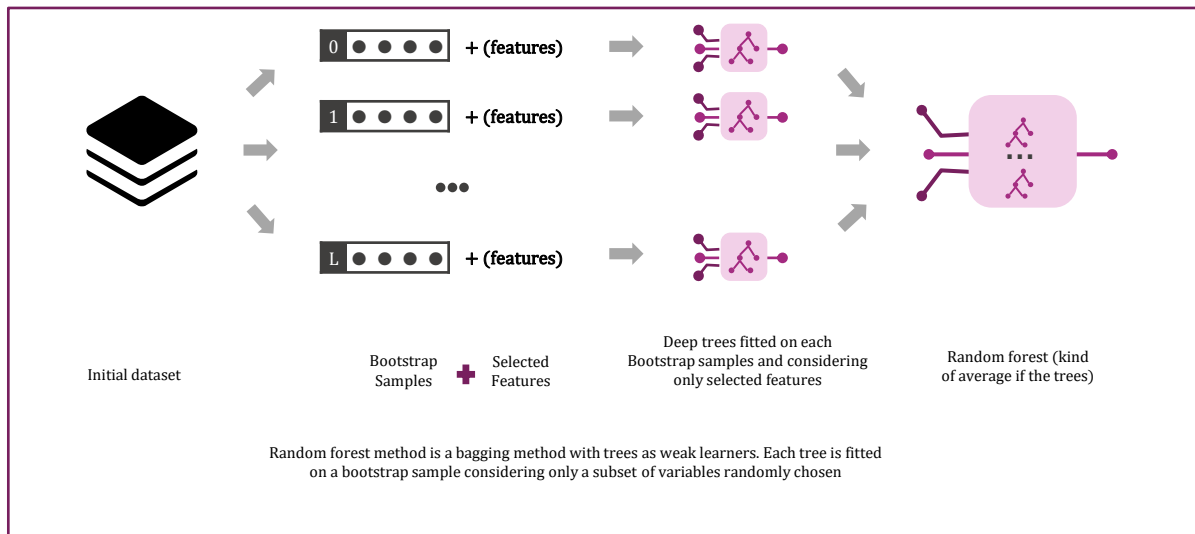


Bagging Example

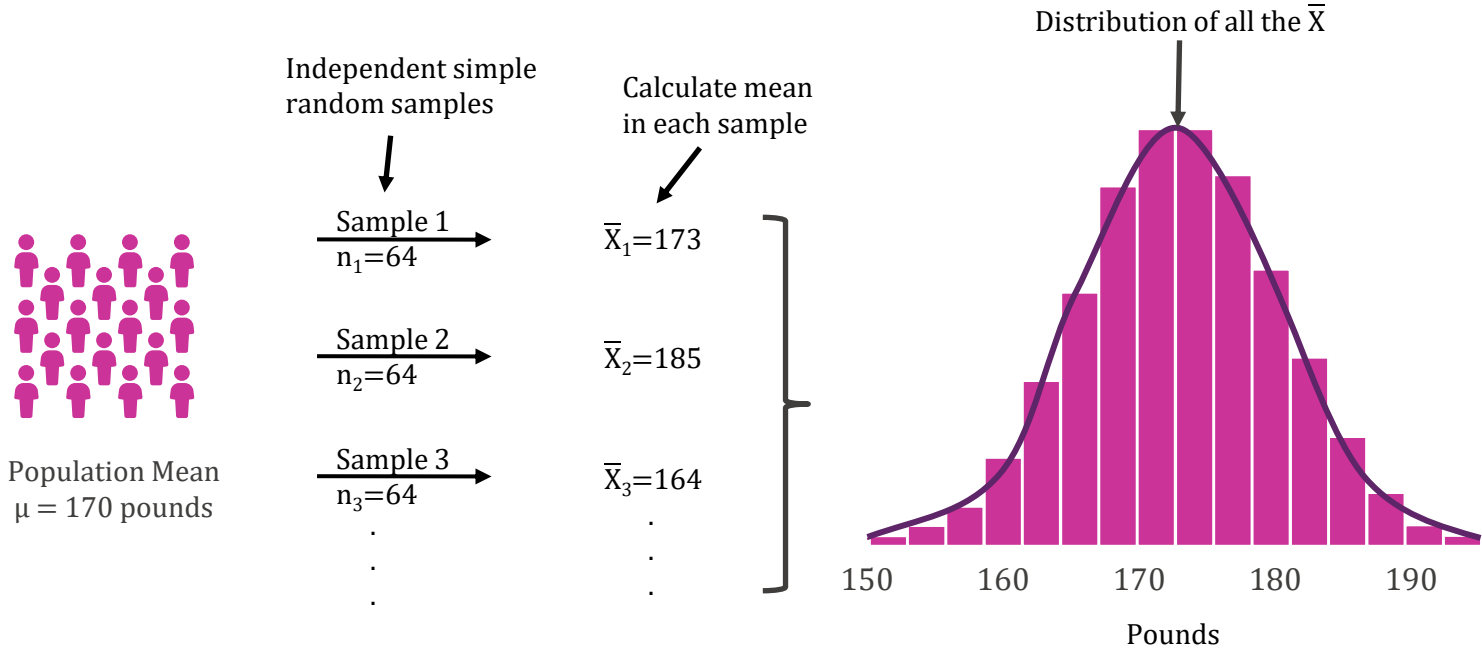


Random Forest

- It is a type of bagging Decision Tree models with a small tweak which decorrelates the trees and improves performance.
- Decision Trees are prone to overfit due to high variance so Random Forests are used to overcome this shortcoming.
- Decision Trees(“weak learners”) are grown fully over each bootstrap samples.
- To decorrelate the trees with bootstrapping datapoints, dataset features are also sampled and a random subset of features is only kept to build the tree.



Sampling Distribution of Mean and Central Limit Theorem (Revisited)



Maximum Features

- Let us say the dimension of your data is 50 and the Maximum Features is 10, each time you need to find the split, you randomly select 10 features and use them to decide which one of the 10 is the best feature to use. When you go to the next node you will select randomly another 10 and so on.
- This mechanism along with bootstrapping data points is used to control overfitting by decorrelating the trees.
- We can either specify a number to denote the Maximum Features at each split or a fraction to denote the percentage of features to consider while making a split.
- We also have options such as $\sqrt{\cdot}$, $\log_2$, etc.

Random Forest - Hyperparameters

Maximum Leaf Nodes

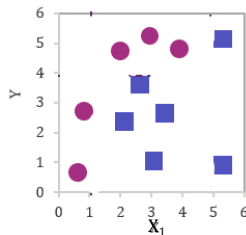
- It represents the maximum number of leaf nodes each of the trees (“weak learners”) in the Random Forest.
- When this hyperparameter is specified, we won’t get full grown trees but pruned trees having lesser number of leaf nodes with minimal effect on the impurity of the tree.
- If None value is passed, the trees (“weak learners”) in the forest can have any number of leaf nodes.

Minimum Impurity Decrease

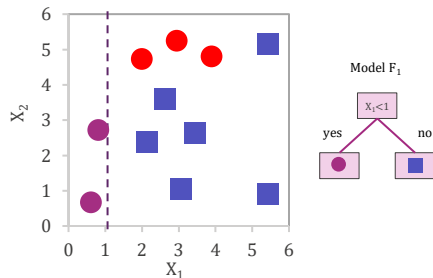
- Like Minimum Sample Split and Minimum Sample Leaf this hyperparameter also controls splitting at a certain node.
- A split is made only if the decrease in the impurity exceeds a certain threshold value provided in this hyperparameter.
- This also helps in preventing overfitting of the trees, but if too high threshold is provided, it’ll increase the overall bias of the trees (“weak learners”).

Boosting

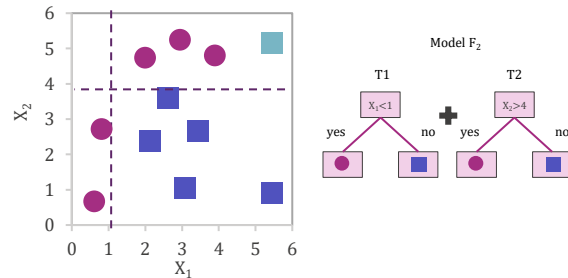
Boosting is a technique that consists in fitting sequentially multiple weak learners in a very adaptative way: each model in the sequence is fitted giving more importance to observations in the dataset that were badly handled by the previous models in the sequence.



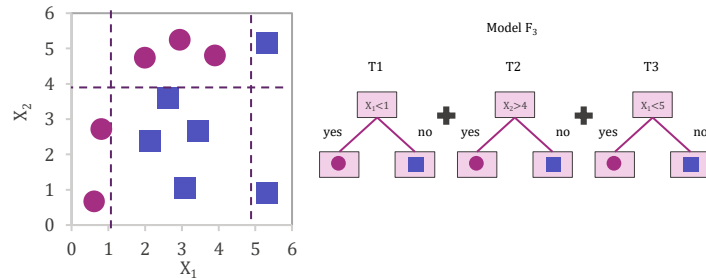
Iteration 1



Iteration 2



Iteration 3



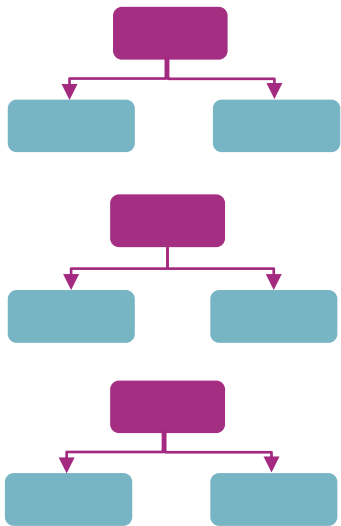
Adaptive Boosting (AdaBoost)

1. Initialize the observation weights $w_i=1/N$, $i= 1, 2, \dots, N$.
2. For $m = 1$ to M repeat steps (a) – (d) :
 - a) Fit a classifier $C_m(x)$ to the training data using weights w_i .
 - b) Compute weighted error of newest tree

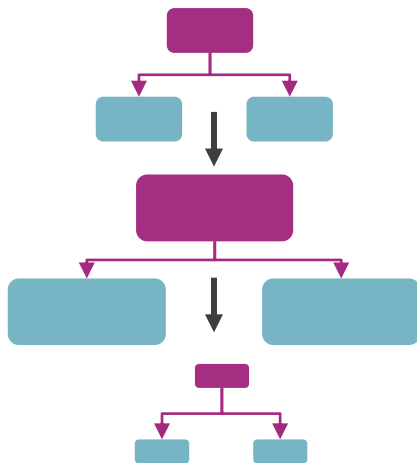
$$\text{err}_m = \frac{\sum_{i=1}^N w_i I(y_i \neq C_m(x_i))}{\sum_{i=1}^N w_i}$$

- c) Compute $\alpha_m = \log[(1 - \text{err}_m)/\text{err}_m]$.
 - d) Update weights for $i=1, \dots, N$: $w_i \leftarrow w_i \cdot \exp[\alpha_m \cdot I(y_i \neq C_m(x_i))]$
and renormalize to w_i to sum to 1.
3. Output $C(x) = \text{sign} [\sum_{m=1}^M \alpha_m C_m(x)]$

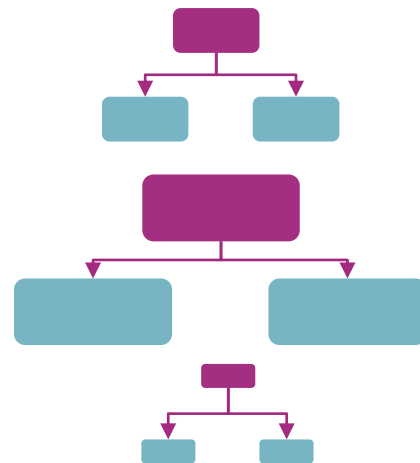
AdaBoost Comparison



Random Forest combines several parallel “weak learners” where each weak learner has equal say or weights



In AdaBoost “weak learner” is made sequentially by taking into account the mistakes made by the previous weak learners



Here some “weak learners” have more say than the other

